



SSE206 计算机网络

1-2章知识点梳理

南雨宏

nanyh@mail.sysu.edu.cn

2026-06-10

高频问题



- Q1 传输时延 vs 传播时延
- Q2 TCP vs UDP 使用场景
- Q3 BitTorrent 传输机制
- Q4 Web 缓存工作原理
- Q5 持续 vs 非持续 HTTP
- Q6 DNS 资源记录类型
- Q7 CDN 两种部署策略
- Q8 P2P vs C/S 架构
- Q9 TCP 三次握手
- Q10 SMTP / POP3 / IMAP
- Q11 TCP/IP 五层 vs OSI 七层
- Q12 排队时延的变化性
- Q13 DASH 自适应流媒体
- Q14 BitTorrent Tit-for-Tat
- Q15 HTTP 无状态与 Cookie
- Q16 进程标识: IP + 端口号
- Q17 存储-转发分组交换

Q1 传输时延 vs 传播时延



■ 传输时延 (Transmission Delay)

- **将分组的所有比特推入链路所需的时间**
- 公式: $d_{\text{trans}} = L / R$
- L = 分组大小 (bit), R = 链路带宽 (bps)
- 取决于: 分组大小、链路带宽; 与物理距离无关

■ 传播时延 (Propagation Delay)

- **单个比特从链路一端传播到另一端的时间**
- 公式: $d_{\text{prop}} = d / s$
- d = 链路物理长度 (m), s 约 2×10^8 m/s (约光速 2/3)
- 取决于: 物理距离; 与分组大小、带宽完全无关

■ 类比: 传输时延 = 收银台结账时间; 传播时延 = 货车在路上的行驶时间

Q2 TCP vs UDP — 如何选择?



■ 为什么 HTTP / SMTP / IMAP 选 TCP

- 要求可靠、有序、无差错传输
- 网页缺图/邮件内容丢失不可接受
- TCP 提供重传、确认、流量控制
- 可靠性大于速度时选 TCP

■ UDP 的优势场景

- 实时音视频 (Zoom、微信语音)：少量丢包可接受，不能容忍重传延迟
- DNS 查询：报文短，速度优先
- 在线游戏：低延迟大于完整性
- DASH / 直播：应用层自行处理丢包

■ 可靠性要求高选 TCP；实时性优先、少量丢包可接受选 UDP

Q3 BitTorrent 新节点如何获得第一个文件块?



■ 问题根源

- 新节点没有任何文件块，无法用 Tit-for-Tat 机制互换
- 陷入「无内容无法获取内容」的死锁

■ 传输机制

- 每隔 30 秒，随机选择一个邻居，无条件向其上传数据块
- 不论对方有无贡献都发送；
- 新节点收到第一个块后，就有了可交换的内容；
- 之后通过 Tit-for-Tat 进入正常互惠流程

■ 关键词：随机选择 + 无条件上传，给新节点机会

Q4 Web 缓存器如何工作?



■ 工作原理

- 浏览器将 HTTP 请求发给 Web 缓存器 (代理服务器)
- 缓存命中: 直接返回副本, 无需访问源服务器, 加速
- 缓存未命中: 向源服务器请求, 存副本, 再转发给客户

■ 为什么Web缓存只能减少部分对象的时延?

- 首次访问 (未命中): 时延 = 缓存查找 + 去源服务器, 可能更长
- 动态内容 / 个性化页面 (如购物车) 无法缓存
- 过期缓存需通过条件 GET (If-Modified-Since) 验证

■ 只有命中的静态内容才真正加速

Q5 持久 HTTP vs 非持久HTTP — RTT 计算



■ 连接方式对比

- 非持久（串行）：每个对象新建 TCP 连接， $9 \times 2\text{RTT} = 18 \text{ RTT}$
- 持久（无流水线）：复用连接，逐个请求， $\text{RTT} + 1\text{RTT} + 8\text{RTT} = 10 \text{ RTT}$
- 持久（流水线）：复用连接，连续发送， 2RTT 最优

■ 时延计算说明

- RTT_0 = 客户端到服务器的往返时延
- 建立 TCP 连接需要 1 RTT (SYN 到 SYN-ACK)
- 发送 HTTP 请求并收到响应需要 1 RTT
- 流水线：将多个请求打包在同一个 RTT 内并发发出

Q6 DNS 资源记录 (RR) 类型



■ 主要记录类型

- A: 主机名 -> IPv4 地址 (最基本的域名解析)
- AAAA: 主机名 -> IPv6 地址
- CNAME: 别名 -> 规范主机名 (如 www 到真实主机名)
- MX: 域名 -> 邮件服务器主机名 (指定收件邮件服务器)
- NS: 域名 -> 权威 DNS 服务器 (指向该域的权威名字服务器)

■ MX 记录的特殊作用

- mail.foo.com 和 www.foo.com 可以拥有完全相同的别名
- 通过 MX 解析到邮件服务器、通过 A/CNAME 解析到 Web 服务器, 互不干扰
- 允许同一机构的邮件与 Web 服务共用相同域名, 灵活分离后端服务器

Q7 CDN 两种服务器放置策略



■ 深入部署 (Enter Deep)

- 部署位置：接入 ISP 内部，靠近用户；节点数量：数万个（高度分散）
- 优点：延迟极低，带宽充足，用户触手可及
- 缺点：维护成本极高，管理复杂
- 代表：Akamai

■ 邀请做客 (Bring Home)

- 部署位置：IXP（互联网交换点）；节点数量：少量大型数据中心
- 优点：节点少，易于管理，运营成本相对较低
- 缺点：离用户较远，延迟较高
- 代表：Google、Netflix

Q8 P2P 架构 vs C/S 架构



■ C/S 架构

- 服务器始终在线，集中式；客户端不直接通信
- 用户增多则服务器压力增大；存在单点故障风险
- 典型应用：Web、邮件、流媒体

■ P2P 架构

- 无专用服务器；对等方直接互通
- 用户增多则上传能力也增加（自扩展）；分散，无单点故障
- 典型应用：BitTorrent、早期 Skype

■ P2P 核心优势：自扩展性

■ 每新增一个节点，既是内容消费者，也是内容供应者（同时上传）

Q9 TCP 三次握手与连接建立时延



■ 三次握手过程

- 客户端到服务器: SYN (请求建立连接)
- 服务器到客户端: SYN-ACK (确认 + 同步序列号)
- 客户端到服务器: ACK (可附带第一个 HTTP 请求)

■ 时延分析

- 前两步合计 = 1 RTT (往返)
- 第三步可与数据包合并发出, TCP 握手开销 = 1 RTT
- HTTP 请求 + 响应 = 1 RTT

■ 获取一个 Web 对象最少需要 2 RTT

■ 持久连接: 后续对象仅 1 RTT

Q10 电子邮件协议：SMTP / POP3 / IMAP



■ 协议对比

- SMTP（推送）：发送邮件，UA到服务器、服务器到服务器，TCP 端口 25
- POP3（拉取）：从服务器下载邮件到本地，简单，下载后默认删除服务器副本
- IMAP（拉取）：在服务器上管理邮件，支持多设备同步，功能丰富

■ 邮件传输完整流程

- 发件人 UA 经 SMTP 到发件服务器，再经 SMTP 到收件服务器，再经 IMAP/POP3 到收件人 UA
- 用户代理与邮件服务器之间：IMAP/POP3（拉取）
- 服务器之间：SMTP（推送）
- 邮件服务器充当存储-转发中继：即使收件方离线，邮件也不会丢失

Q11 TCP/IP 五层模型 vs OSI 七层模型



■ 模型对比

- 应用层（两模型相同）：HTTP, SMTP, DNS, FTP
- 表示层（OSI 独有）：加密、压缩、数据格式转换
- 会话层（OSI 独有）：会话管理、同步
- 传输层（两模型相同）：TCP, UDP
- 网络层（两模型相同）：IP, 路由协议
- 链路层（两模型相同）：以太网, Wi-Fi (MAC 层)
- 物理层（两模型相同）：光纤、铜缆、无线信号

■ 关键差异

- OSI 表示层（加密/压缩）和会话层（同步/管理）
- 在 TCP/IP 中合并到应用层，由应用自行实现

Q12 排队时延为什么是变化的？



■ 排队时延（动态变化）

- 取决于网络实时负载（当前正在等待转发的分组数量）
- 流量强度： $\rho = La / R$ （ L =分组大小， a =到达速率， R =带宽）
- ρ 趋近 1 时，排队时延趋近无穷大； ρ 大于 1 时，队列无限增长
- **表现为网络拥塞；无法提前精确预测**

■ 其他三种时延（相对固定）

- 处理时延：路由器读首部 + 差错检验，取决于硬件，通常小于毫秒
- 传输时延： $d_{\text{trans}} = L / R$ ，由分组大小和链路带宽决定，可精确计算
- 传播时延： $d_{\text{prop}} = d / s$ ，由物理距离决定，与网络负载无关

Q13 DASH 自适应流媒体的工作机制



■ DASH 工作原理

- 视频切成若干小块 (chunk)，每块有多种码率版本，存储于不同 URL
- 客户端实时测量可用带宽
- 自动选择合适码率：带宽好则高清；带宽差则标清
- 动态自适应，无需用户手动干预

■ 核心优势

- 用户侧自适应，无需服务器感知每个客户端状态
- 同一内容多种质量，单服务器支持所有设备
- 缓冲区管理：避免卡顿与不必要的低清播放
- 客户端完全掌控选择逻辑，灵活可扩展

Q14 BitTorrent Tit-for-Tat（一报还一报）



■ 机制说明

- 每个节点监测邻居对自己的上传速率并排名
 - 优先向给自己上传最快的 4 个邻居回馈上传（奖励机制）
 - 每隔 10 秒重新评估，更新优先服务名单
 - 若对方停止上传则自己也停止对其上传（对等制衡）
-
- 核心目的：防止搭便车（只下载不上传），激励用户贡献上传带宽
 - 每 30 秒：随机选一个额外邻居无条件上传
 - 给新节点机会，同时让老节点探索更好的合作伙伴

Q15 HTTP 无状态特性与 Cookie 机制



■ HTTP 为什么是无状态的？

- 服务器不保存任何关于客户端的历史请求信息
- 每次 HTTP 请求独立处理
- 设计目的：简化服务器设计，提高可扩展性
- 问题：无法记住登录状态，无法实现个性化服务

■ Cookie 如何维持状态？

- 首次访问：服务器响应头设置 Set-Cookie: id=1678
- 浏览器本地存储该 Cookie
- 后续请求：请求头自动携带 Cookie: id=1678
- 服务器查数据库识别用户；购物车、登录态均靠此实现

■ Cookie 并未破坏 HTTP 无状态性——无状态是协议本身特性；Cookie 是应用层的外挂式状态管理

Q16 进程标识：IP 地址 + 端口号



- 唯一标识网络中另一台主机上的进程，需要两个信息
 - IP 地址：标识网络上的哪台主机（全球唯一）
 - 端口号：标识该主机上的哪个进程/应用（主机内唯一）
 - 套接字 (Socket) = IP 地址 + 端口号，是应用层与运输层之间的接口

- 常见知名端口
 - HTTP: 80 (Web 访问明文) ; HTTPS: 443 (加密 Web)
 - SMTP: 25 (发送邮件) ; DNS: 53 (域名解析)

- 注意
 - 同一主机两个浏览器访问同一网站：IP 相同，OS 为每个连接临时分配不同端口号
 - 端口范围：0 - 65535；知名端口：0 - 1023

Q17 存储-转发分组交换机制



■ 工作原理

- 路由器必须收到完整分组后才开始向下一条链路转发
- 过程：接收，缓存，读首部，差错检验，转发
- 端到端时延（N 段链路）： $d = N \times (L / R)$ （忽略传播时延）

■ 为什么必须等整个分组到达？

- 原因 1：需要读取首部信息
- 目的 IP 决定转发路径；不完整分组无法读取首部
- 原因 2：需要差错校验
- CRC 需要完整数据才能运算；不完整分组无法通过检验

■ 协议设计要求，非技术限制，保证转发的正确性